

PRACTICAL 4

Name: Shweta Hangal

PRN: 202501050004

4.1.1 Pandas Series creation and Manipulation

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the groupby and mean() operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Refer to the sample test cases for better understanding regarding input and output format.

Code:

```
import pandas as pd
```

```
# Take inputs from the user to create a list of numbers
```

```
numbers = list(map(int, input().split()))
```

```
# Create a Pandas series from the list of numbers
```

```
series = pd.Series(numbers)
```

```
# Grouping by even and odd numbers and calculating the mean
```

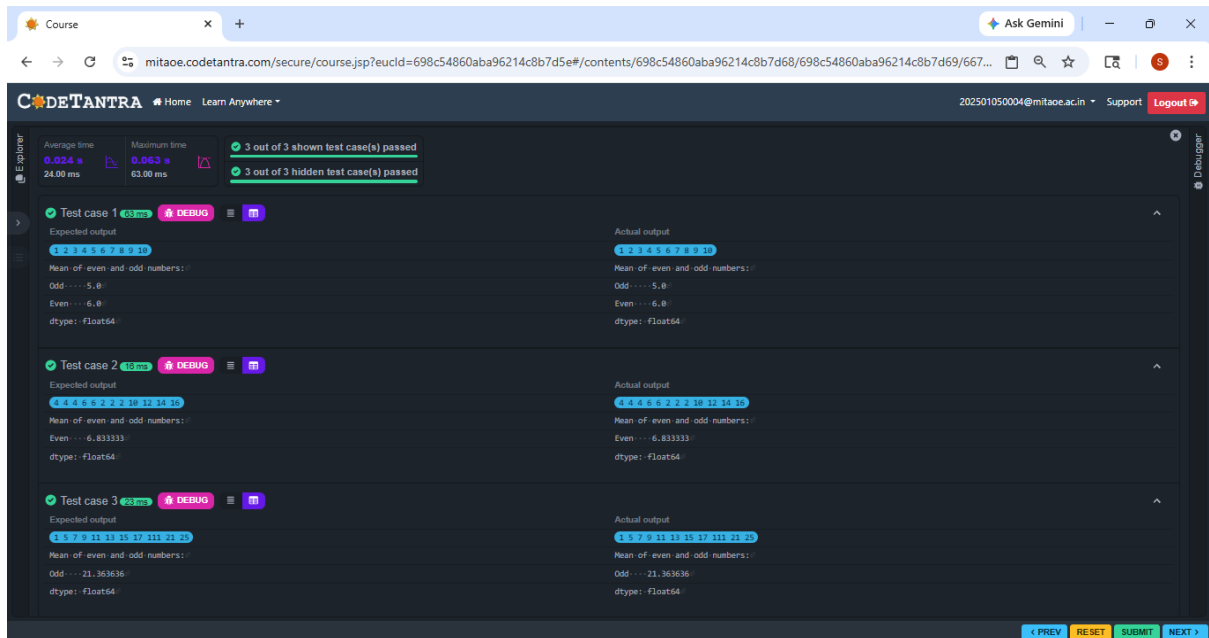
```
grouped = series.groupby(series % 2 == 0).mean()
```

```
# Display the mean of even and odd numbers with labels
```

```
grouped.index = ['Even' if is_even else 'Odd' for is_even in grouped.index]
```

```
print("Mean of even and odd numbers:")
```

```
print(grouped)
```



4.1.2 Dictionary to Dataframe

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.

- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column Gender with values taken from the user.
- Display the DataFrame after adding the new column.

Modify a column:

- Convert names to uppercase.
- Display the DataFrame after modifying the column.

Delete a column:

- Remove the Age column.
- Display the DataFrame after deleting the column.

Code:

```
import pandas as pd
```

```
# Provided dictionary of lists
```

```
data = {  
    'Name': ['Alice', 'Bob', 'Charlie'],  
    'Age': [25, 30, 35],  
}
```

```
# Convert the dictionary to a DataFrame
```

```
df = pd.DataFrame(data)
```

```
# Display the original DataFrame
```

```
print("Original DataFrame:")
```

```
print(df)
```

```
# Adding a new row
```

```
new_name = input("New name: ")
```

```
new_age = int(input("New age: "))
```

```
df.loc[len(df)] = [new_name, new_age]
```

```
# Display the DataFrame after adding a new row
```

```
print("After adding a row:\n",df)
```

```
# Modifying a row
```

```
row_to_modify = int(input("Index of row to modify: "))
```

```
new_age_value = int(input("New age: "))
```

```
df.at[row_to_modify, "Age"] = new_age_value
```

```
# Display the DataFrame after modifying a row
```

```
print("After modifying a row:")
```

```
print(df)
```

```
# Deleting a row
```

```
row_to_delete = int(input("Index of row to delete: "))
```

```
df = df.drop(index=row_to_delete).reset_index(drop=True)
```

```
# Display the DataFrame after deleting a row
```

```
print("After deleting a row:")
```

```
print(df)
```

```
# Adding a new column
```

```
genders = input("Enter genders separated by space: ").split()
```

```
df["Gender"]=genders
```

```
# Display the DataFrame after adding a new column
```

```
print("After adding a new column:")
```

```
print(df)
```

```
# Modifying a column
```

```
df["Name"] = df["Name"].str.upper()
```

```
# Display the DataFrame after modifying a column
```

```
print("After modifying a column:")
```

```
print(df)
```

```
# Deleting a column
```

```
df = df.drop(columns=["Age"])
```

```
# Display the DataFrame after deleting a column
```

```
print("After deleting a column:")
```

```
print(df)
```

The screenshot shows the CoderTantra IDE interface. At the top, there's a browser tab with the URL `mitaoe.codetantra.com/secure/course.jsp?euclid=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d68/698c54860aba96214c8b7d69/667...`. The IDE header includes the CoderTantra logo, navigation links (Home, Learn Anywhere), a user profile (202501050004@mitaoe.ac.in), and a Logout button. The main workspace is divided into two panes: Explorer on the left and Debugger on the right. The Explorer pane shows a file named `Test case 1` with a status of `1 out of 1 shown test case(s) passed` and `1 out of 1 hidden test case(s) passed`. The Debugger pane displays the execution flow and output. The output shows the original DataFrame with columns 'Name' and 'Age', and a new row being added for 'Susan' with age 29. The final DataFrame after adding the row is shown.

```
Original DataFrame:
  Name  Age
0  Alice  25
1  Bob   30
2  Charlie 35
New name: Susan
New age: 29
After adding a row:
  Name  Age
0  Alice  25
1  Bob   30
2  Charlie 35
3  Susan  29
Index of row to modify: 3
New age: 29
After modifying a row:
  Name  Age
0  Alice  27
1  Bob   30
2  Charlie 35
3  Susan  29
```

The screenshot shows the CoderTantra IDE interface. At the top, there's a browser tab with the URL `mitaoe.codetantra.com/secure/course.jsp?euclid=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d68/698c54860aba96214c8b7d69/667...`. The IDE header includes the CoderTantra logo, navigation links (Home, Learn Anywhere), a user profile (202501050004@mitaoe.ac.in), and a Logout button. The main workspace is divided into two panes: Explorer on the left and Debugger on the right. The Explorer pane shows a file named `Test case 1` with a status of `1 out of 1 shown test case(s) passed` and `1 out of 1 hidden test case(s) passed`. The Debugger pane displays the execution flow and output. The output shows the DataFrame after deleting a row and adding a new column 'Gender'. The final DataFrame after adding the new column is shown.

```
After modifying a row:
  Name  Age
0  Alice  27
1  Bob   30
2  Charlie 35
3  Susan  29
Index of row to delete: 3
After deleting a row:
  Name  Age
0  Alice  27
1  Bob   30
2  Charlie 35
Enter genders separated by space: F M
After adding a new column:
  Name  Age Gender
0  Alice  27  F
1  Bob   30  M
2  Charlie 35  M
After modifying a column:
  Name  Age Gender
0  ALICE  27  F
1  BOB   30  M
2  CHARLIE 35  M
After deleting a column:
  Name  Age
0  ALICE  27
1  BOB   30
2  CHARLIE 35
```

4.1.3 Student Information

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).

- Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').

Note:

Refer to the displayed test cases for better understanding.

Code:

```
import pandas as pd

# Read the text file into a DataFrame
file = input()

data = pd.read_csv(file, sep="\s+", header=None, names=["Name", "Age", "Grade"])

print("First five rows:")

print(data.head())

av = round(data['Age'].mean(),2)

print("Average age:", av)

fd=data[data['Grade'].isin(['A','B'])]

print("Students with a grade up to B")

print(fd)
```

The screenshot shows the CodeTANTRA IDE interface. At the top, there's a navigation bar with 'Course', 'Home', and 'Learn Anywhere' links. The main workspace displays the code execution results for a test case named 'Test case 1'. The 'Expected output' and 'Actual output' sections both show the same results: 'First five rows:' followed by a table of student data (Name, Age, Grade) and 'Students with a grade up to B:' followed by another table of student data. The status bar at the bottom indicates '1 out of 1 shown test case(s) passed' and '1 out of 1 hidden test case(s) passed'. Navigation buttons for 'PREV', 'RESET', 'SUBMIT', and 'NEXT' are visible at the bottom right.

4.2.1 Month with the Highest Total Scores

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Code:

```
import pandas as pd

# Prompt the user for the file name
file_name = input()

# Load the data
```

```

df = pd.read_csv(file_name)

df["Date"] = pd.to_datetime(df["Date"])

df["Month"] = df["Date"].dt.to_period("M")

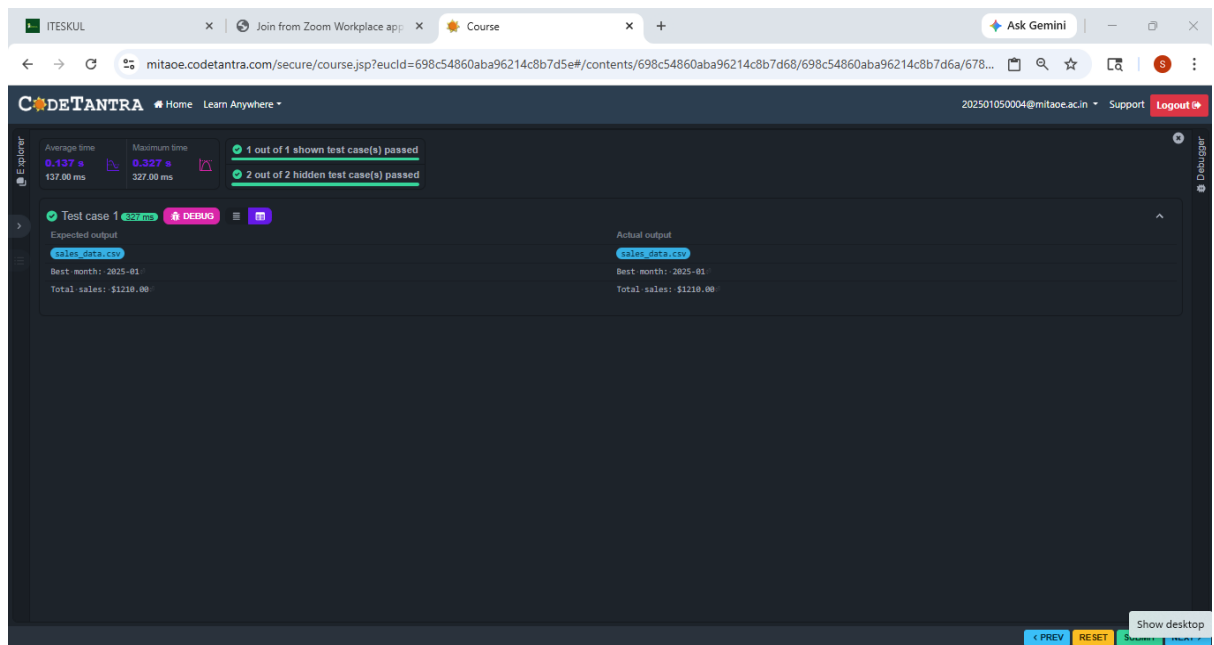
df["Total Sales"] = df["Quantity"] * df["Price"]

monthly_sales = df.groupby("Month")["Total Sales"].sum()

# Find the month with the highest total sales
best_month = monthly_sales.idxmax()
highest_sales = monthly_sales.max()

print(f"Best month: {best_month}")
print(f"Total sales: ${highest_sales:.2f}")

```



4.2.2 Best Selling Product

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.

- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

Sample Data:

Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Code:

```
import pandas as pd

# Prompt the user for the file name
file_name = input()

# Load the data
df = pd.read_csv(file_name)
ps=df.groupby('Product')['Quantity'].sum()

# Find the product with the highest total quantity sold
```

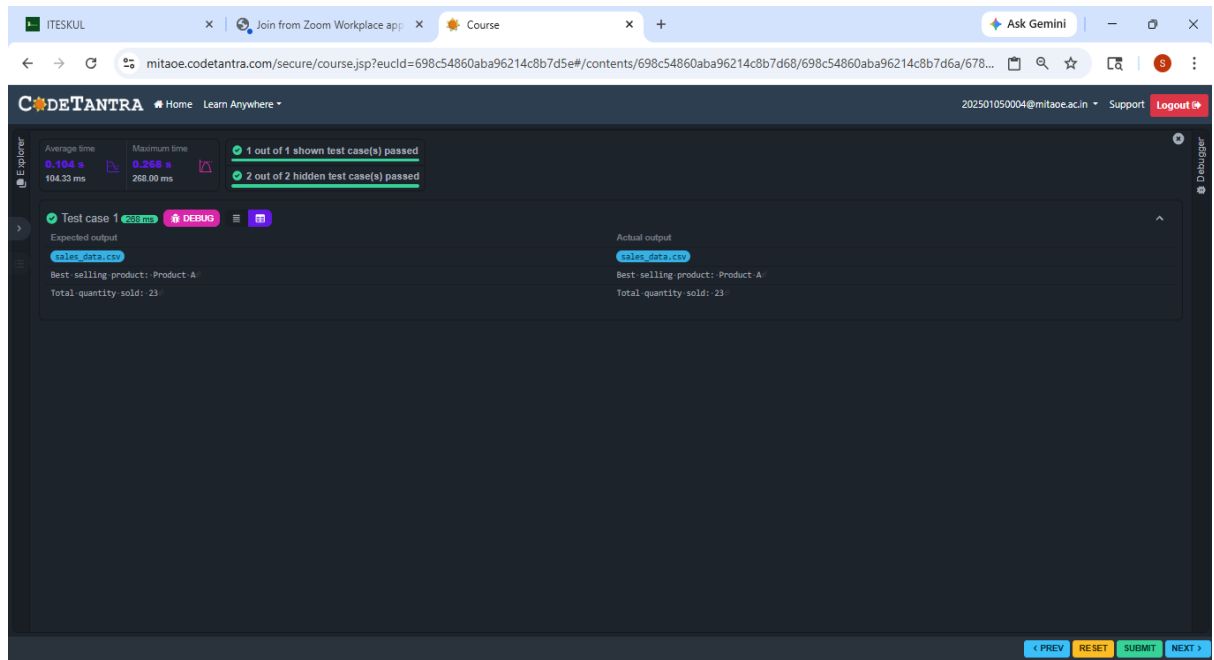
```
best_product = ps.idxmax()
```

```
highest_quantity = ps.max()
```

```
# Display the result
```

```
print(f"Best selling product: {best_product}")
```

```
print(f"Total quantity sold: {highest_quantity}")
```



4.2.3 City that sold the most Products

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Sample Data:

Date,Product,Quantity,Price,City

2025-01-01,Product A,5,20,New York

2025-01-01,Product B,3,15,Los Angeles

2025-01-02,Product A,7,20,New York

2025-01-02,Product C,4,30,Chicago

2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Code:

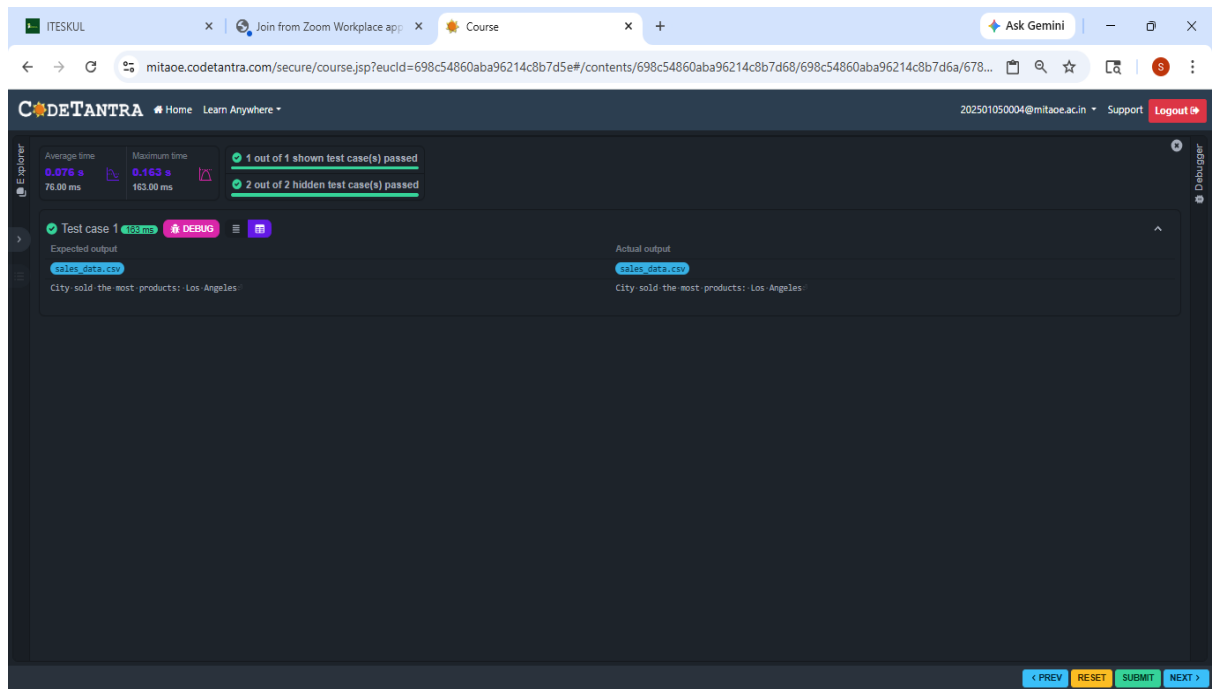
```
import pandas as pd

# Prompt the user for the file name
file_name = input()

# Load the data
df = pd.read_csv(file_name)

# write the code..
ps=df.groupby('City')['Quantity'].sum()
best_city=ps.idxmax()

# Display the result
print(f"City sold the most products: {best_city}")
```



4.2.4 Most Frequently Sold Product Pairs

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pair/s that was sold most frequently.

Sample Data:

Date,Product,Quantity,Price,City

2025-01-01,Product A,5,20,New York

2025-01-01,Product B,3,15,Los Angeles

2025-01-02,Product A,7,20,New York

2025-01-02,Product C,4,30,Chicago

2025-01-03,Product B,2,15,Chicago

2025-01-03,Product A,8,20,Los Angeles

2025-01-04,Product C,6,30,New York

2025-01-04,Product B,5,15,Los Angeles

2025-01-05,Product A,3,20,Chicago

2025-01-05,Product C,10,30,Los Angeles

Explanation:

Transactions:

- 2025-01-01: Product A, Product B
- 2025-01-02: Product A, Product C
- 2025-01-03: Product B, Product A
- 2025-01-04: Product C, Product B
- 2025-01-05: Product A, Product C

Now, let's count how often the pairs of products appear together:

- Product A and Product B: Appear in transactions on 2025-01-01 and 2025-01-03.
- Product A and Product C: Appear in transactions on 2025-01-02 and 2025-01-05.
- Product B and Product C: Appears in transactions on 2025-01-04.

Most Frequent Product Combinations:

- Product A and Product B (2 times)
- Product A and Product C (2 times)

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Code:

```
import pandas as pd

from itertools import combinations

from collections import Counter


# Prompt user to input the file name
file_name = input()


# Read data from the specified CSV file
df = pd.read_csv(file_name)
```

```
# write the code
```

```
grouped = df.groupby("Date")["Product"].apply(list)
```

```
pairs_counter = Counter()
```

```
for products in grouped:
```

```
    pairs = combinations(sorted(products), 2)
```

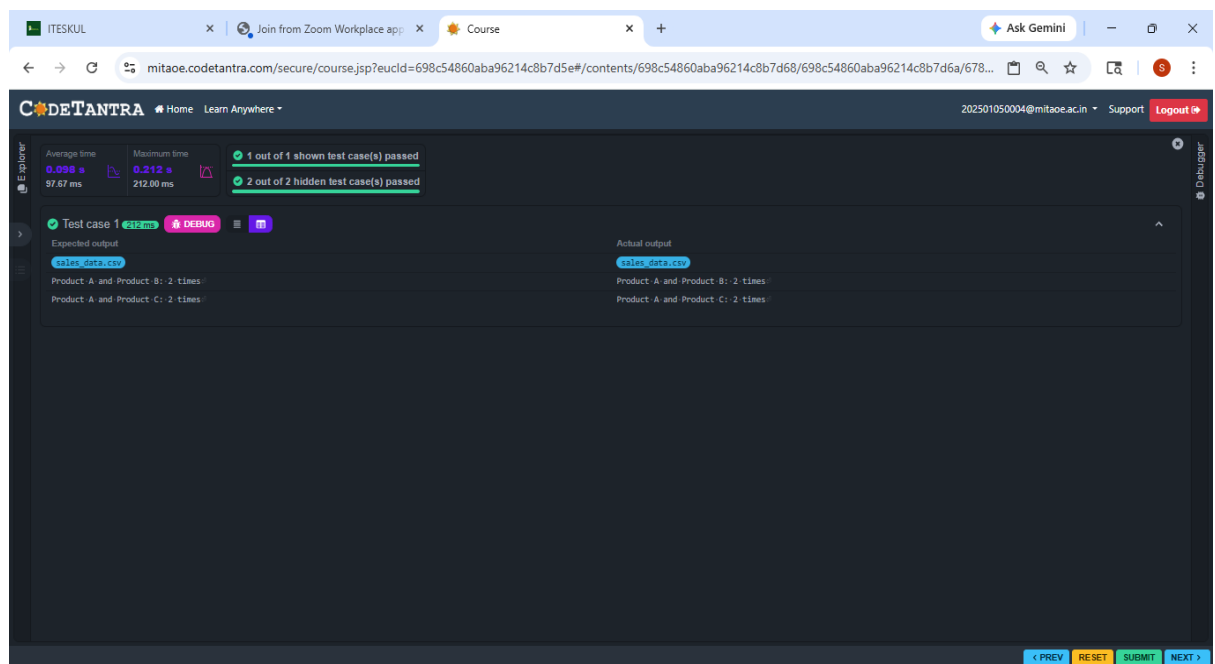
```
    pairs_counter.update(pairs)
```

```
max_frequency = max(pairs_counter.values())
```

```
most_common_pairs = [(pair, freq) for pair, freq in pairs_counter.items() if freq ==  
max_frequency]
```

```
for (product1, product2), frequency in most_common_pairs:
```

```
    print(f"{product1} and {product2}: {frequency} times")
```



4.2.5 Titanic Dataset Analysis and Data Cleaning

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).

4. Get a summary of the dataset (using .info()).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using .describe().
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (mode()).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	

Sample Data:

PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked

1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S

2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC
17599,71.2833,C85,C

3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S

4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S

5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S

6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q

7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S

8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S

9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina
Berg)",female,27,0,2,347742,11.1333,,S

10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.

Code:

```
import pandas as pd
import numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# 1. Display the first 5 rows of the dataset
print(data.head())

# 2. Display the last 5 rows of the dataset
print(data.tail())

# 3. Get the shape of the dataset
print(data.shape)

# 4. Get a summary of the dataset (info)
print(data.info())

# 5. Get basic statistics of the dataset
print(data.describe())

# 6. Check for missing values
print(data.isnull().sum())

# 7. Fill missing values in the 'Age' column with the median age
median_age = data['Age'].median()
data['Age'].fillna(median_age, inplace=True)

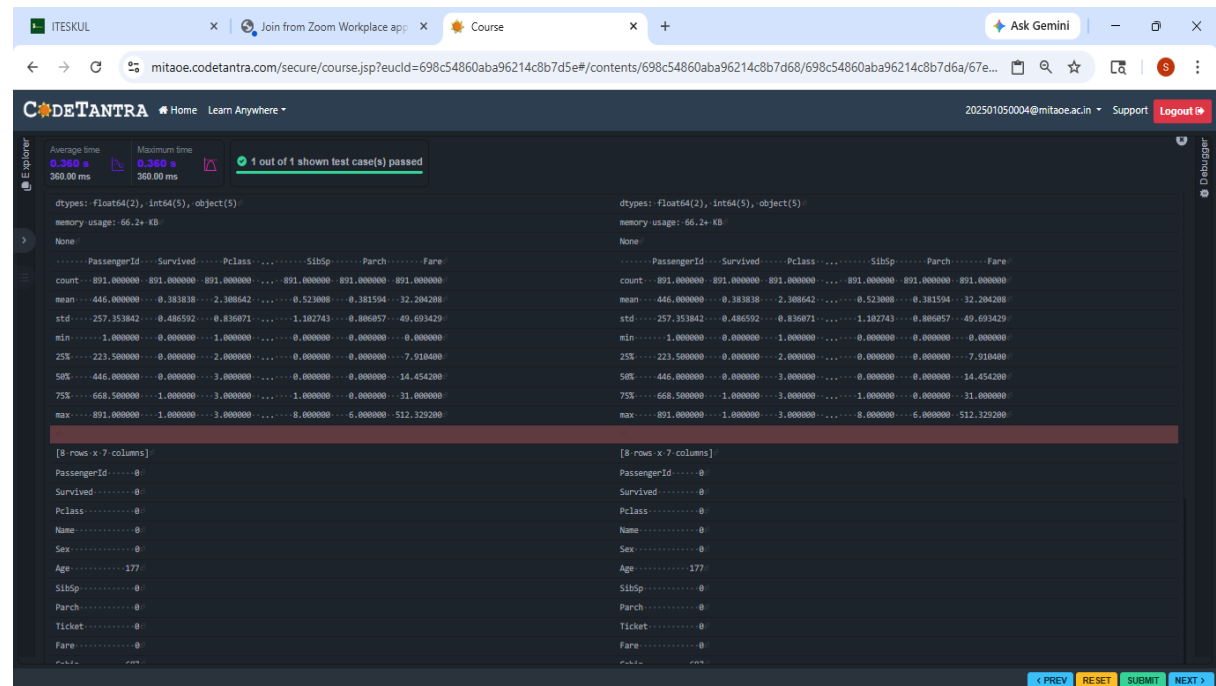
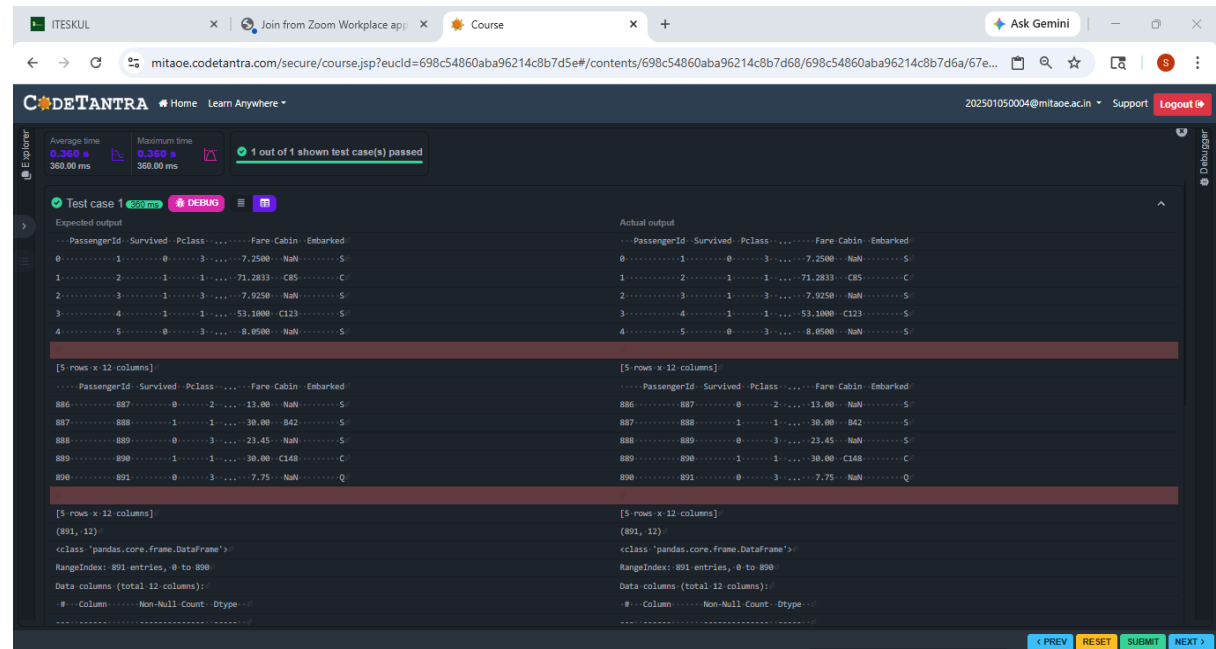
# 8. Fill missing values in the 'Embarked' column with the mode
```

```
mode_embarked = data['Embarked'].mode() [0]
```

```
data['Embarked'].fillna(mode_embarked, inplace=True)
```

9. Drop the 'Cabin' column due to many missing values

```
data.drop('Cabin', axis=1, inplace=True)
```



4.2.6 Titanic Dataset Analysis and Data Cleaning 2

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	

Sample Data:

PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked

1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S

2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC
17599,71.2833,C85,C

3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S

4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S

5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S

6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q

7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S

8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S

9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina
Berg)",female,27,0,2,347742,11.1333,,S

10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.

Code:

```
import pandas as pd
import numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] + data['Parch']
# 1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)
data['Alone'] = np.where(data['FamilySize']==0,1,0)

# 2. Convert 'Sex' to numeric (male: 0, female: 1)
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})

# 3. One-hot encode the 'Embarked' column
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# 4. Get the mean age of passengers
mean_age = data['Age'].mean()
print(mean_age)

# 5. Get the median fare of passengers
median_fare = data['Fare'].median()
print(median_fare)

# 6. Get the number of passengers by class
```

```
passengers_by_class = data['Pclass'].value_counts()
print(passengers_by_class)
```

7. Get the number of passengers by gender

```
passengers_by_gender = data['Sex'].value_counts().sort_index()
print(passengers_by_gender)
```

8. Get the number of passengers by survival status

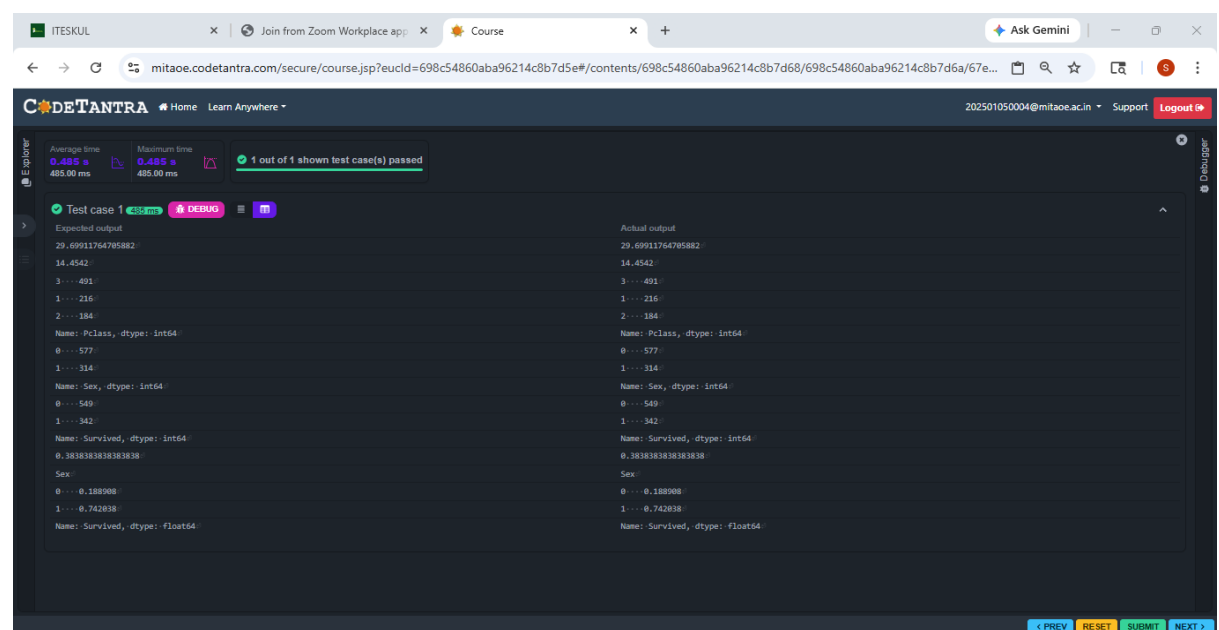
```
passengers_by_survival = data['Survived'].value_counts().sort_index()
print(passengers_by_survival)
```

9. Calculate the survival rate

```
survival_rate = data['Survived'].mean()
print(survival_rate)
```

10. Calculate the survival rate by gender

```
gender_survival = data.groupby('Sex')['Survived'].mean()
print(gender_survival)
```



The screenshot shows a web browser window with the URL mitaoe.codetantra.com/secure/course.jsp?euclid=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d68/698c54860aba96214c8b7d6a/67e.... The page is titled "CODETANTRA" and shows a test case result for a Python program. The test case is labeled "Test case 1" and shows "Expected output" and "Actual output" side-by-side. The expected output is a pandas Series with values 29, 14, 3, 1, 2 for categories 0, 1, 1, 2, 2. The actual output is a pandas Series with values 29, 14, 3, 1, 2 for categories 0, 1, 1, 2, 2. The test case is marked as "passed".

Expected output	Actual output
29.69911764795882	29.69911764795882
14.4542	14.4542
3.491	3.491
1.216	1.216
2.184	2.184

Test case 1 passed

Expected output

```
29.69911764795882
14.4542
3.491
1.216
2.184
Name: Pclass, dtype: int64
0.577
1.314
Name: Sex, dtype: int64
0.549
1.342
Name: Survived, dtype: int64
0.3838383838383838
Sex
0.188989
1.742838
Name: Survived, dtype: float64
```

Actual output

```
29.69911764795882
14.4542
3.491
1.216
2.184
Name: Pclass, dtype: int64
0.577
1.314
Name: Sex, dtype: int64
0.549
1.342
Name: Survived, dtype: int64
0.3838383838383838
Sex
0.188989
1.742838
Name: Survived, dtype: float64
```

4.2.7 Titanic Dataset Analysis and Data Cleaning 3

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	

Sample Data:

PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked

1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S

2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC
17599,71.2833,C85,C

3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S

4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S

5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S

6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q

7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S

8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S

9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S

10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.

Code:

```
import pandas as pd
```

```
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
```

```
data['FamilySize'] = data['SibSp'] + data['Parch']
```

```
data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
```

```
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
```

```
# 1. Calculate the survival rate by class
```

```
print(data.groupby('Pclass')['Survived'].mean())
```

```
# 2. Calculate the survival rate by embarked location
```

```
print(data.groupby('Embarked_S')['Survived'].mean())
```

```
# 3. Calculate the survival rate by family size
```

```
print(data.groupby('FamilySize')['Survived'].mean())
```

```
# 4. Calculate the survival rate by being alone
```

```
print(data.groupby('IsAlone')['Survived'].mean())
```

```
# 5. Get the average fare by class
```

```
print(data.groupby('Pclass')['Fare'].mean())
```

6. Get the average age by class

```
print(data.groupby('Pclass')['Age'].mean())
```

7. Get the average age by survival status

```
print(data.groupby('Survived')['Age'].mean())
```

8. Get the average fare by survival status

```
print(data.groupby('Survived')['Fare'].mean())
```

9. Get the number of survivors by class

```
print(data[data['Survived'] == 1]['Pclass'].value_counts())
```

10. Get the number of non-survivors by class

```
print(data[data['Survived'] == 0]['Pclass'].value_counts())
```

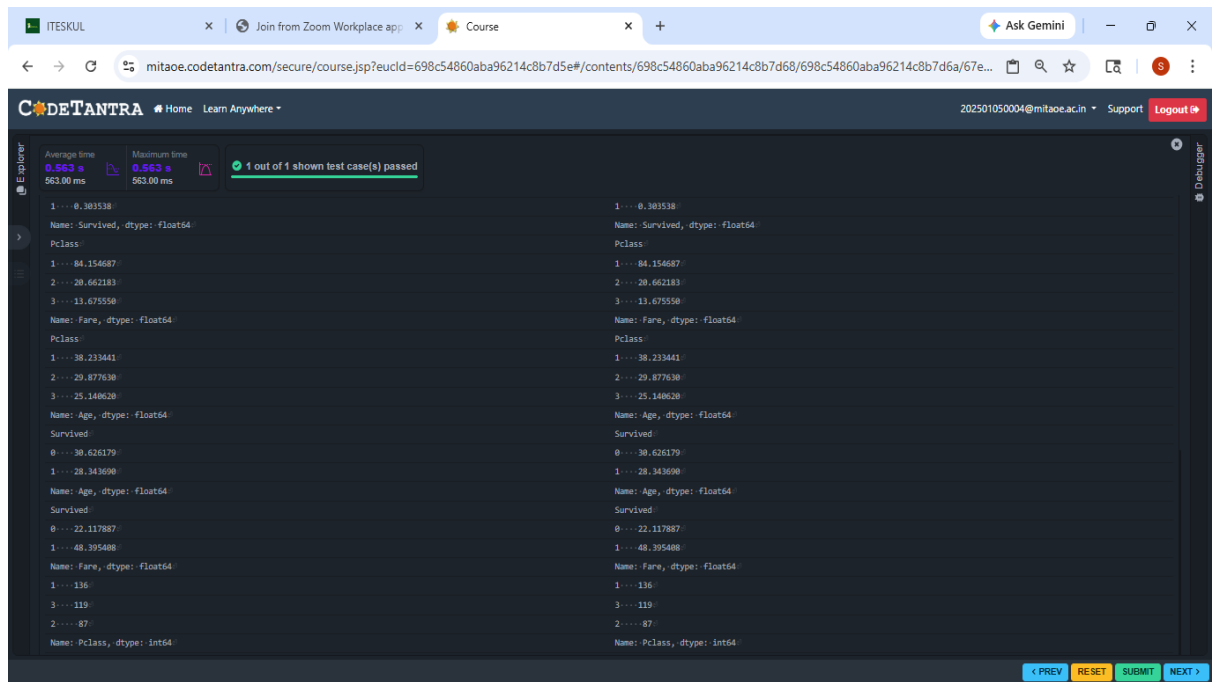
The screenshot displays the CodeTANTRA web IDE interface. At the top, there's a navigation bar with 'Home' and 'Learn Anywhere' links. The main workspace shows a Python script with the following content:

```
print(data.groupby('Pclass')['Age'].mean())
```

The output of the script is displayed in the 'Actual output' section, showing the mean age for each passenger class:

```
Pclass
1... 0.629630
2... 0.471826
3... 0.242363
```

The 'Expected output' section shows the same results, indicating that the test case passed. The interface also includes a 'DEBUG' button and a '1 out of 1 shown test case(s) passed' status message. At the bottom, there are navigation buttons: 'PREV', 'RESET', 'SUBMIT', and 'NEXT'.



4.2.8 Titanic Dataset Analysis and Data Cleaning 4

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).
2. Get the number of non-survivors by gender (Sex).
3. Get the number of survivors by embarkation location (Embarked_S).
4. Get the number of non-survivors by embarkation location (Embarked_S).
5. Calculate the percentage of children (Age < 18) who survived.
6. Calculate the percentage of adults (Age >= 18) who survived.
7. Get the median age of survivors.
8. Get the median age of non-survivors.
9. Get the median fare of survivors.
10. Get the median fare of non-survivors.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
-------------	----------	--------	------	-----	-----	-------	-------	--------

--	--	--	--	--	--	--	--	--	--

Sample Data:

PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked

1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S

2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC
17599,71.2833,C85,C

3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S

4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S

5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S

6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q

7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S

8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S

9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina
Berg)",female,27,0,2,347742,11.1333,,S

10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.

Code:

```
import pandas as pd
```

```
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
```

```
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
```

```
# 1. Get the number of survivors by gender
```

```
survivors_by_gender = data[data['Survived'] == 1]['Sex'].value_counts()
```

```
print(survivors_by_gender)
```

2. Get the number of non-survivors by gender

```
non_survivors_by_gender = data[data['Survived'] == 0]['Sex'].value_counts()  
print(non_survivors_by_gender)
```

3. Get the number of survivors by embarked location

```
survivors_by_embarked_s = data[data['Survived'] == 1]['Embarked_S'].value_counts()  
print(survivors_by_embarked_s)
```

4. Get the number of non-survivors by embarked location

```
non_survivors_by_embarked_s = data[data['Survived'] ==  
0]['Embarked_S'].value_counts()  
print(non_survivors_by_embarked_s)
```

5. Calculate the percentage of children (Age < 18) who survived

```
children = data[data['Age'] < 18]  
children_survival_rate = children['Survived'].mean()  
print(children_survival_rate)
```

6. Calculate the percentage of adults (Age >= 18) who survived

```
adults = data[data['Age'] >= 18]  
adults_survival_rate = adults['Survived'].mean()  
print(adults_survival_rate)
```

7. Get the median age of survivors

```
median_age_survivors = data[data['Survived'] == 1]['Age'].median()  
print(median_age_survivors)
```

8. Get the median age of non-survivors

```
median_age_non_survivors = data[data['Survived'] == 0]['Age'].median()  
print(median_age_non_survivors)
```

9. Get the median fare of survivors

```
median_fare_survivors = data[data['Survived'] == 1]['Fare'].median()

print(median_fare_survivors)
```

10. Get the median fare of non-survivors

```
median_fare_non_survivors = data[data['Survived'] == 0]['Fare'].median()

print(median_fare_non_survivors)
```

The screenshot shows the CODETANTRA web IDE interface. At the top, there's a navigation bar with 'Home' and 'Learn Anywhere' links. The main area displays a test case result for 'Test case 1'. The 'Expected output' and 'Actual output' columns show identical results, indicating the test passed. The output includes several lines of text, such as 'female ... 233', 'male ... 189', 'Name: Sex, dtype: int64', 'male ... 468', 'female ... 81', 'Name: Sex, dtype: int64', '1 ... 217', '0 ... 125', 'Name: Embarked_S, dtype: int64', '1 ... 427', '0 ... 122', 'Name: Embarked_S, dtype: int64', '0.5398230888495575', '0.3818316139767855', '28.0', '28.0', '26.0', and '18.5'. The bottom of the interface features a 'DEBUG' button and a 'PREV' button.

Test case 1
Expected output
female ... 233
male ... 189
Name: Sex, dtype: int64
male ... 468
female ... 81
Name: Sex, dtype: int64
1 ... 217
0 ... 125
Name: Embarked_S, dtype: int64
1 ... 427
0 ... 122
Name: Embarked_S, dtype: int64
0.5398230888495575
0.3818316139767855
28.0
28.0
26.0
18.5

Actual output
female ... 233
male ... 189
Name: Sex, dtype: int64
male ... 468
female ... 81
Name: Sex, dtype: int64
1 ... 217
0 ... 125
Name: Embarked_S, dtype: int64
1 ... 427
0 ... 122
Name: Embarked_S, dtype: int64
0.5398230888495575
0.3818316139767855
28.0
28.0
26.0
18.5

